Guide : Nginx comme Reverse Proxy pour des conteneurs

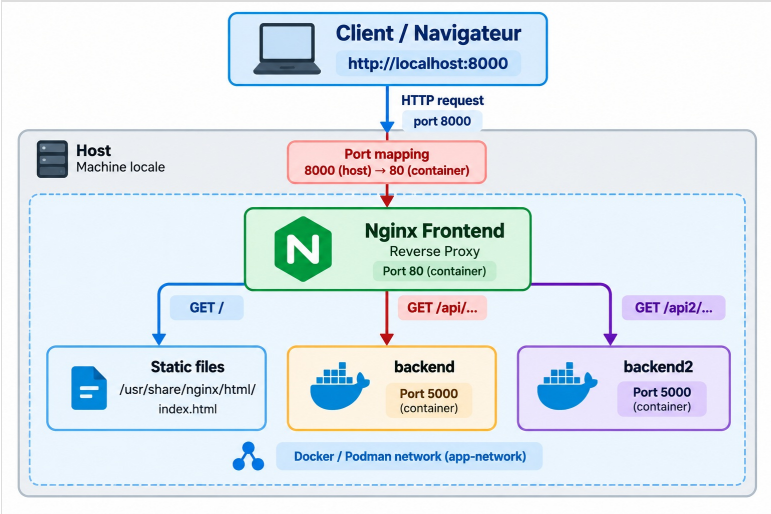
1. Objectif

Nginx peut être utilisé comme **serveur web et reverse proxy** devant plusieurs applications exécutées dans des conteneurs.

Il reçoit les requêtes HTTP des clients, analyse leur URL et les transmet au conteneur approprié selon les règles définies dans sa configuration.

Dans notre architecture, Nginx sert les fichiers statiques du frontend et route les requêtes API vers les backends.

Exemple d'architecture



Les conteneurs doivent être connectés au même réseau Podman ou Docker pour que Nginx puisse joindre les backends en utilisant leurs noms.

Principe important : le navigateur communique avec Nginx. Nginx communique ensuite avec les backends. Les clients n'ont donc pas besoin d'accéder directement aux ports des applications backend.

---

2. Les fichiers et répertoires importants

Chemin ou commande	Rôle
`/usr/share/nginx/html/`	Répertoire par défaut contenant les fichiers statiques du site, notamment `index.html`.
`/etc/nginx/conf.d/default.conf`	Fichier de configuration du serveur Nginx dans l'image officielle Nginx.
`nginx -g 'daemon off;'`	Démarre Nginx en premier plan, ce qui permet au processus de rester le processus principal du conteneur.

`nginx -s reload`	Demande à Nginx de recharger sa configuration sans arrêter complètement le service.

Dans un conteneur, Nginx est généralement lancé en premier plan pour que le moteur de conteneurs puisse suivre son processus principal.

Pour recharger la configuration, exécutez la commande dans le conteneur :

```
docker exec frontend nginx -s reload
```

Remplacez `frontend` par le nom réel du conteneur Nginx.

> *Si vous modifiez la configuration sur l'hôte, assurez-vous qu'elle est montée dans le conteneur ou intégrée à une nouvelle image. Sinon, le rechargement ne prendra pas en compte les modifications effectuées uniquement sur l'hôte.*

---

3. Comprendre la configuration Nginx

Voici un exemple de configuration qui sert un frontend et route deux chemins API vers deux backends différents.

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location /api/ {
        proxy_pass http://backend:5000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api2/ {
        proxy_pass http://backend2:5000/api2/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location / {
        try_files $uri $uri/ =404;
    }
}
```

3.1. Le bloc `server`

```
server {
    listen 80;
```

```
    server_name _;
}
```

- `listen 80;` : Nginx écoute les requêtes HTTP sur le port 80 du conteneur.
- `server_name _;` : configuration utilisée comme serveur par défaut dans cet exemple, selon la sélection des blocs `server` par Nginx.

3.2. Le frontend statique

```
root /usr/share/nginx/html;
index index.html;
```

- `root` indique le répertoire dans lequel Nginx recherche les fichiers statiques.
- `index index.html;` définit le fichier d'index à utiliser lorsqu'une requête vise un répertoire.

Par exemple, une requête vers `/` permet de servir :

```
/usr/share/nginx/html/index.html
```

3.3. Le routage avec `location`

Nginx utilise les blocs `location` pour choisir comment traiter une requête en fonction de son URI.

```
location /api/ {
    proxy_pass http://backend:5000/api/;
}
```

Une requête telle que :

```
http://localhost:8080/api/login
```

est transmise à :

```
http://backend:5000/api/login
```

Le nom `backend` est le nom du conteneur ou un nom DNS résolvable sur le réseau des conteneurs. Le port `5000` est le port sur lequel l'application backend écoute **à l'intérieur du conteneur**.

De même :

```
location /api2/ {
    proxy_pass http://backend2:5000/api2/;
}
```

Une requête vers `/api2/login` est transmise à `backend2`, sur son port `5000`, avec le chemin `/api2/login`.

Enfin :

```
location / {
    try_files $uri $uri/ =404;
}
```

Ce bloc sert les fichiers statiques du frontend. Si le fichier ou le répertoire demandé n'existe pas, Nginx renvoie une erreur HTTP `404`.

---

4. Comment Nginx choisit-il le conteneur ?

Le routage repose sur les chemins définis dans les blocs `location`.

Requête du client	Destination	Rôle
`/`	Fichiers du frontend	Afficher la page web
`/index.html`	Fichiers du frontend	Afficher le fichier HTML
`/api/login`	`backend:5000/api/login`	Traiter la connexion via le backend
`/api/users`	`backend:5000/api/users`	Accéder à une API du premier backend
`/api2/login`	`backend2:5000/api2/login`	Transmettre la requête au deuxième backend

Le navigateur envoie toutes ces requêtes à la même adresse publique, par exemple `http://localhost:8080`. Nginx décide ensuite de la destination interne.

Le rôle de `proxy_pass`

La directive `proxy_pass` définit l'adresse du serveur backend vers lequel Nginx transmet la requête.

```
proxy_pass http://backend:5000/api/;
```

Décomposition :

- `http://` : protocole utilisé entre Nginx et le backend.
- `backend` : nom du service ou du conteneur résolvable sur le réseau.
- `5000` : port interne du backend.
- `/api/` : chemin utilisé dans l'URI transmise.

---

5. Comprendre les en-têtes HTTP

Dans les blocs `location`, plusieurs directives complètent la transmission des requêtes.

```
proxy_http_version 1.1;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
```

Directive	Explication
`proxy_http_version 1.1;`	Utilise HTTP/1.1 pour la communication avec le backend.
`proxy_set_header Host $host;`	Transmet l'en-tête `Host` correspondant au nom d'hôte demandé par le client.

`proxy_set_header X-Real-IP $remote_addr;`	Transmet l'adresse IP du client vue par Nginx dans l'en-tête `X-Real-IP`.

Ces en-têtes permettent notamment au backend de récupérer des informations utiles sur la requête d'origine.

---

6. Pourquoi utiliser le nom du conteneur plutôt que son adresse IP ?

Il est préférable d'utiliser :

```
proxy_pass http://backend:5000/api/;
```

plutôt que :

```
proxy_pass http://10.89.0.40:5000/api/;
```

L'adresse IP d'un conteneur peut changer lorsqu'il est supprimé puis recréé. Le nom du conteneur permet d'éviter de coder en dur cette adresse, à condition que la résolution de noms fonctionne sur le réseau partagé.

Par exemple, si Nginx tente de joindre `10.89.0.40` alors que le backend possède désormais l'adresse `10.89.0.42`, les requêtes peuvent échouer et produire une erreur `502 Bad Gateway`.

Avec un réseau de conteneurs correctement configuré, le nom `backend` permet de joindre le conteneur sans dépendre d'une adresse IP fixe.

---

7. Tester et dépanner

Vérifier les conteneurs

```
docker ps
```

Vérifier la configuration Nginx

```
docker exec frontend nginx -t
```

Cette commande vérifie la syntaxe de la configuration.

Afficher la configuration réellement chargée

```
docker exec frontend nginx -T
```

Cette commande permet de repérer une ancienne configuration, notamment une adresse IP codée en dur.

Recharger la configuration

```
docker exec frontend nginx -s reload
```

Consulter les logs

```
docker logs frontend
docker logs backend
docker logs backend2
```

Comprendre les erreurs courantes

Erreur	Cause possible
`502 Bad Gateway`	Backend inaccessible, application arrêtée, mauvais port ou mauvais nom.
`403 Forbidden`	Accès refusé par Nginx ou l'application backend, à cause des permissions, des règles d'accès ou de la configuration.
`404 Not Found`	Chemin inexistant ou route API non définie dans le backend.
`Host is unreachable`	Problème de réseau ou destination inaccessible.
`Connection refused`	Aucun service n'écoute à l'adresse et au port visés, ou la connexion est refusée.

Une erreur `502` ne signifie pas forcément que le fichier Nginx est incorrect : il faut aussi vérifier que le backend est opérationnel et joignable depuis le conteneur frontend.

---

8. À retenir

- Nginx peut servir les fichiers statiques et fonctionner comme reverse proxy.
- `root` et `index` définissent où se trouvent les fichiers du frontend.
- `location` sélectionne le traitement à appliquer selon l'URI.
- `proxy_pass` transmet la requête à un backend précis.
- Les noms des conteneurs peuvent être utilisés à la place d'adresses IP fixes si la résolution DNS du réseau fonctionne.
- `nginx -t` vérifie la configuration, `nginx -T` l'affiche et `nginx -s reload` demande son rechargement.

En résumé : le navigateur communique avec Nginx, puis Nginx route chaque requête vers les fichiers du frontend ou vers le backend approprié en fonction du chemin demandé.